Software Design Analysis Report

Smart Parking Management System (SPMS) | Prepared using the supplied A-G report template

A. Project Details

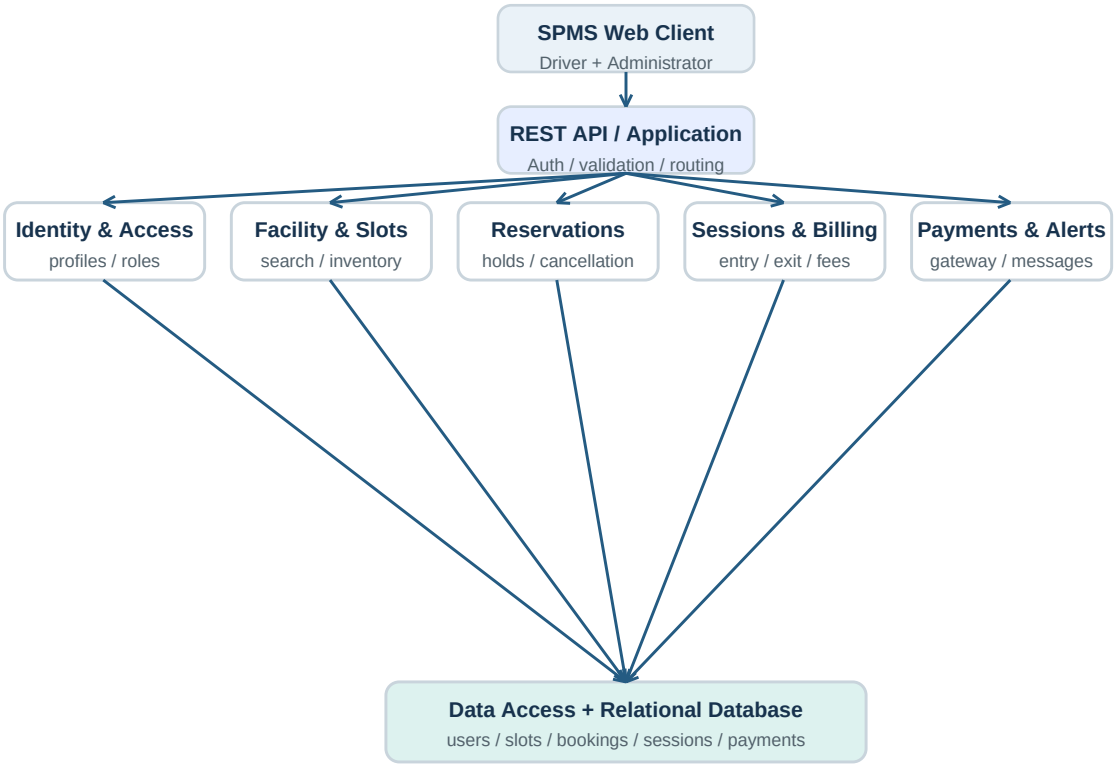
Project detail	SPMS design
Project title	Smart Parking Management System (SPMS)
Team / roll numbers	Anubhav Mittal, Amit Kumar, Ankit Modi, Arpit Gupta. Roll numbers: _____
Project objective	Enable drivers to find and reserve parking, while automating facility slot management, vehicle entry/exit tracking, fee calculation, payment, and operational reporting.
Technology stack	React / HTML / CSS / JavaScript client; Node.js / Express REST API; MySQL or PostgreSQL relational database; payment gateway and IoT/ANPR integrations.

B. Module Identification and Modularization

Total number of modules: 7. The system is divided by cohesive business responsibility and integration boundary. This keeps user-facing concerns, core parking rules, persistence, and external providers independently understandable.

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	Identity & Access	Registration, authentication, profiles, role checks	Credentials/profile to session or access result	M2, M7
M2	Facility & Slot Management	Facility, slot, type, availability and pricing configuration	Search/configuration to facility and slot data	M7
M3	Reservation Management	Search, temporary hold, confirm, cancel	Driver + slot + time to reservation state	M2, M4, M7
M4	Parking Session & Billing	Entry/exit events, session state, fee calculation	Validated events + tariff to fee/session	M2, M5, M7
M5	Payment Processing	Gateway requests, results, receipts/refunds	Amount + idempotency key to payment status	M6, M7
M6	Notifications & Reporting	Booking alerts, occupancy and revenue summaries	Domain events / records to message/report	M7
M7	Persistence & Integration	Repositories, transactions, payment/sensor adapters	Domain commands to database/provider protocols	Database and external systems

Module hierarchy / decomposition diagram



External integrations: Payment Gateway / IoT / ANPR / Notification provider

The client sends requests to the application API. Domain modules handle focused use cases and access shared records through persistence interfaces. External systems are reached through adapters; they do not directly update the database.

C. Design Strategy

Selected strategy: Hybrid top-down and bottom-up.

Top-down decomposition starts from the parking workflow and separates identity, facility/slot management, reservations, sessions/billing, payments, and reporting. Bottom-up integration reuses common building blocks such as authentication middleware, repository interfaces, transaction handling, and provider adapters.

Justification: The system has clear user goals and business capabilities, which are easy to decompose from the overall workflow. It also relies on reusable technical components and external services that are best integrated as stable interfaces. Combining both strategies supports traceability from requirements while avoiding duplicated infrastructure.

Evidence: Driver search to reserve to park to pay to exit is decomposed into domain modules. Reusable data access, identity, notification, and integration components are assembled into those workflows.

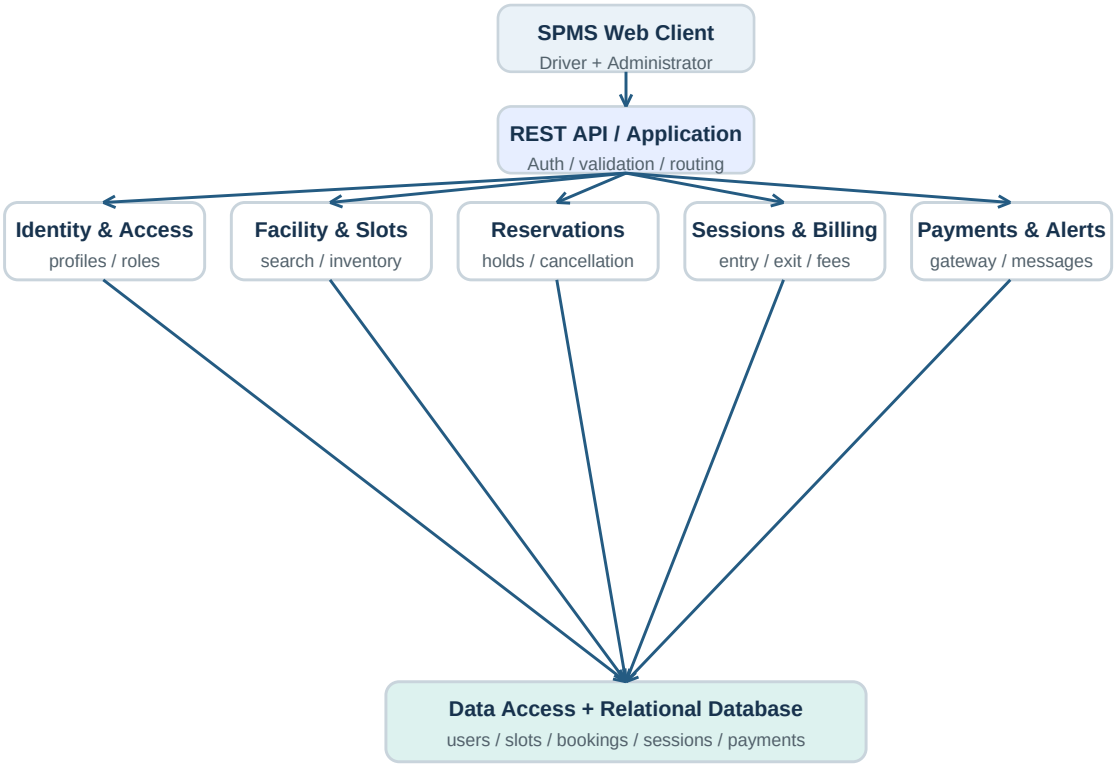
D. Design Approach

Selected approach: Mixed (Object-Oriented domain model + function-oriented orchestration).

Object-oriented design represents long-lived domain concepts and their relationships: User, Facility, ParkingSlot, Reservation, ParkingSession, Payment, PricingRule, and EntryExitEvent. Services expose focused operations such as reserveSlot, startSession, calculateFee, and processPayment.

This combination keeps data and lifecycle rules near the domain entities while making application workflows explicit and testable. Controllers translate HTTP requests; services coordinate use cases; repositories handle database operations. External APIs are isolated behind adapters.

Structure chart / function-oriented view



External integrations: Payment Gateway / IoT / ANPR / Notification provider

The chart shows decomposition from the client/API into business modules, with shared persistence below them. The UML class diagram in *UML-Design.md* provides the complementary object-oriented view.

E. Coupling Analysis

The target is low coupling: modules exchange small, explicit data structures through service and repository interfaces.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
Reservation to Facility/Slot	Availability query and slot identifier	Data coupling / Low	Use availability interface; do not expose slot table internals.
Reservation to Payment	Amount, reservation ID, idempotency key, payment result	Data coupling / Low	Use PaymentPort interface and normalized result model.
Session/Billing to Sensor adapter	Entry/exit event ID, type, timestamp, vehicle reference	Data coupling / Low	Map provider payload to canonical event; deduplicate by event ID.
Reporting to Persistence	Aggregated occupancy and revenue query results	Data coupling / Low	Use reporting queries/read models; avoid direct write access to domain tables.

F. Cohesion Analysis

High cohesion is preferred: each module should have one focused purpose, and its internal tasks should contribute to that purpose.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
Identity & Access	Authenticate users and enforce role permissions	Functional / High	Keep profile/authentication concerns here; leave booking policy to reservation module.
Reservation Management	Find candidates, hold slots, confirm/cancel bookings	Functional / High	Separate reporting and payment-provider logic behind interfaces.
Parking Session & Billing	Track a stay and compute its fee from timestamps and tariff	Sequential-functional / High	Keep tariff calculation as a focused domain service and test boundary rules.
Notifications & Reporting	Send user-facing updates and produce admin summaries	Communicational / Moderate	Separate delivery channels from report aggregation as usage grows.

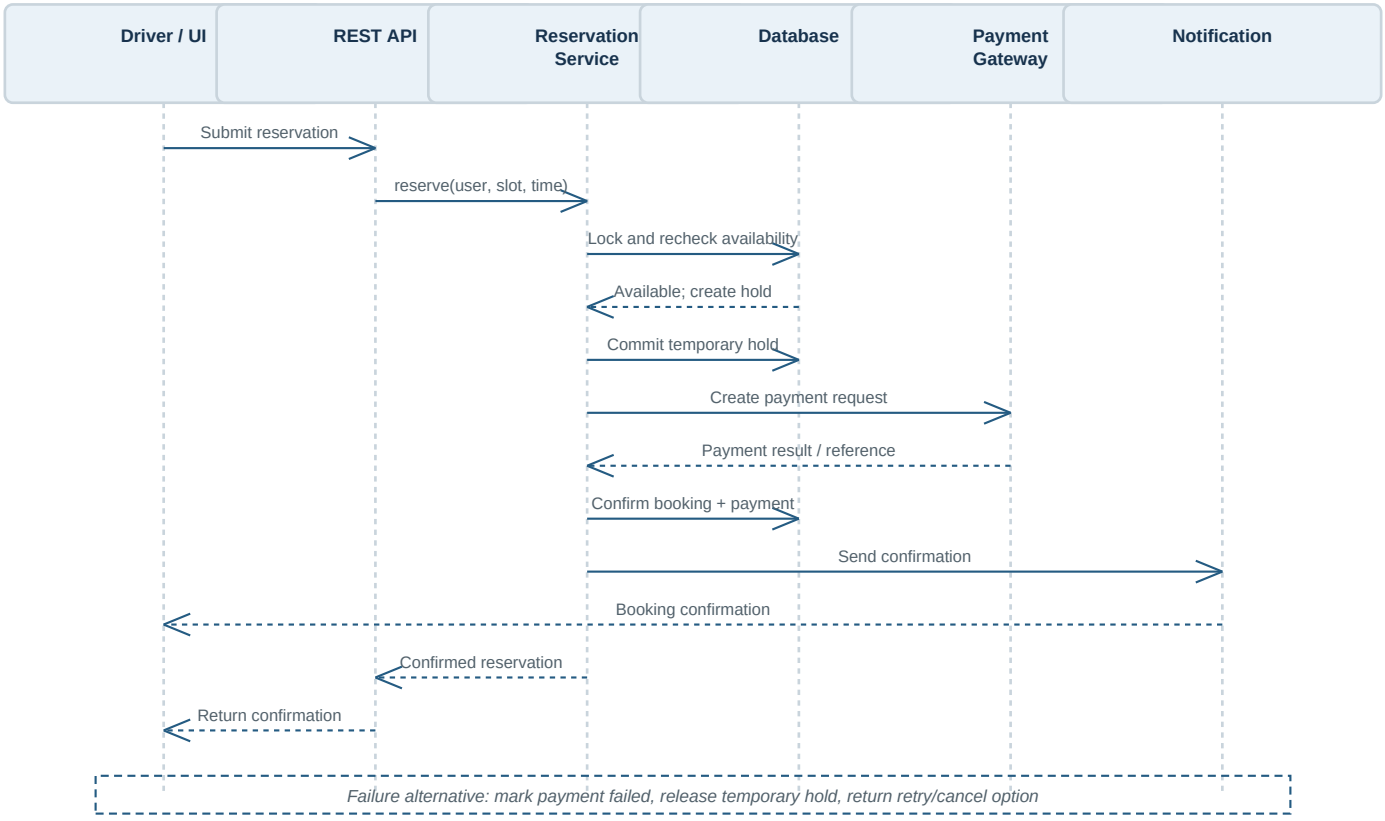
Design quality observations

Quality attribute	Design assessment	Design response
Maintainability	Clear layer and module boundaries support focused change.	Use service interfaces and keep provider-specific code in adapters.
Reliability	Reservation and slot state must stay consistent under concurrency.	Recheck within a transaction; use locks/constraints and explicit states.
Security	Roles, payment callbacks, and sensor events cross trust boundaries.	Enforce server-side RBAC; verify signatures; protect secrets and sensitive data.
Scalability	Availability reads and concurrent booking writes have different needs.	Index search fields; keep API stateless; measure contention before scaling.
Recoverability	External callbacks may be duplicated, delayed, or lost.	Idempotency, event records, bounded retries, and reconciliation workflow.

Coupling and cohesion conclusion: keep business responsibilities strongly focused within each module and make module communication depend on explicit contracts. This supports changes to payment or sensor vendors without rewriting reservation and parking rules.

G. Module Interaction and Design Quality

Sequence diagram: reservation hold and payment confirmation



The reservation service first performs a short transactional availability check and creates a temporary hold. It calls the gateway after committing that hold, then records the confirmed payment and reservation in a second transaction. A failed or delayed payment follows an explicit release or pending/reconciliation path.

Design analysis

The design addresses the principal project risks at their ownership boundaries. Reservation Management owns slot allocation and must prevent overlapping confirmed bookings. Parking Session & Billing owns time-based charges. Payment Processing owns provider communication and payment state. Sensor events are authenticated, recorded, and applied idempotently.

The proposed modular server and relational store keep critical reservation state changes straightforward to transact. Adapters isolate volatile provider protocols. The principal trade-off is shared-database coupling between modules; repository/service interfaces and ownership rules reduce direct dependencies and make future extraction possible.

Validation priorities

1. Concurrent attempts for the same slot and interval result in at most one confirmed reservation.
2. Payment success, decline, timeout, duplicate callback, and delayed webhook produce consistent reservation/payment states.
3. Duplicate or out-of-order sensor events do not create duplicate sessions or release an occupied slot incorrectly.
4. Role permissions, pricing boundaries, cancellation behavior, and reports match the requirements.

Conclusion: The SPMS design uses hybrid decomposition, a mixed object-oriented and service-oriented approach, low-coupling interfaces, and high-cohesion business modules. These choices support understandable workflows, transactional correctness, and maintainable integration with payment and parking hardware.